```
/*-------------------------------------------
  STEP 0: Housekeeping and File Paths
-------------------------------------------*/
options nodate nonumber mprint mlogic symbolgen;
ods graphics on;

%let basepath = /home/u59438703/CINJ;

/*-------------------------------------------
  STEP 1: Import Datasets
-------------------------------------------*/
proc import datafile="&basepath/fqhc_sites.csv" out=fqhc_sites dbms=csv replace;
    getnames=yes;
run;

proc import datafile="&basepath/survey_data.xlsx" out=survey_data dbms=xlsx replace;
    getnames=yes;
run;

/*-------------------------------------------
  STEP 2: Clean FQHC Site Data
-------------------------------------------*/
data fqhc_clean;
    set fqhc_sites;
    lat_num = input(lat, best32.);
    lon_num = input(lon, best32.);
    population_num = input(compress(population, ','), best32.);
    pcp_count_num = input(pcp_count, best32.);

    if missing(lat_num) or missing(lon_num) or missing(population_num) or missing(pcp_count_num) then delete;
run;

/*-------------------------------------------
  STEP 3: Compute Distance Matrix (Haversine)
-------------------------------------------*/
data distance_matrix;
    set fqhc_clean(rename=(lat_num=site_lat lon_num=site_lon));
    retain earth_radius 6371;

    do i = 1 to nobs;
        set fqhc_clean(keep=tract_id lat_num lon_num population_num rename=(lat_num=tract_lat lon_num=tract_lon populat:
        delta_lat = (site_lat - tract_lat) * constant('pi') / 180;
        delta_lon = (site_lon - tract_lon) * constant('pi') / 180;

        a = sin(delta_lat / 2)**2 + cos(tract_lat * constant('pi') / 180) * cos(site_lat * constant('pi') / 180) * sin(
        c = 2 * arsin(min(1, sqrt(a)));
        distance_km = earth_radius * c;

        /* Prevent 0 distance for weight calculation */
        if distance_km < 0.1 then distance_km = 0.1;

        /* Tiered Power Decay */
        if distance_km <= 10 then beta = 1;
        else if distance_km <= 20 then beta = 1.5;
        else if distance_km <= 30 then beta = 2;
        else beta = .;

        if not missing(beta) then decay_weight = 1 / (distance_km**beta);
        else decay_weight = 0;

        weighted_pop = pop * decay_weight;

        output;
    end;
    drop i;
run;

/*-------------------------------------------
  STEP 4: Compute Weighted Demand (D_j) and Supply Ratio (R_j)
-------------------------------------------*/
proc sql;
    create table demand as
    select site_id,
           sum(weighted_pop) as weighted_demand
    from distance_matrix
    group by site_id;
```

```
quit;

proc sql;
    create table site_supply as
    select a.*, b.weighted_demand,
           case
             when b.weighted_demand > 0 then a.pcp_count_num / b.weighted_demand
             else .
           end as Rj
    from fqhc_clean as a
    left join demand as b
    on a.site_id = b.site_id;
quit;

/*-------------------------------------------
  STEP 5: Merge Rj into Distance Matrix
-------------------------------------------*/
proc sql;
    create table distance_with_Rj as
    select a.*, b.Rj
    from distance_matrix as a
    left join site_supply as b
    on a.site_id = b.site_id;
quit;

/*-------------------------------------------
  STEP 6: Compute Accessibility Score (A_i) by Tract
-------------------------------------------*/
data valid_distances;
    set distance_with_Rj;
    if not missing(Rj) and not missing(decay_weight);
run;

proc sql;
    create table accessibility_scores_tiered as
    select tract_id,
           sum(Rj * decay_weight) as accessibility_score
    from valid_distances
    group by tract_id;
quit;

proc export data=accessibility_scores_tiered
    outfile="&basepath/accessibility_scores_tiered.csv"
    dbms=csv replace;
run;

/*-------------------------------------------
  STEP 7: Visualization of Accessibility Scores
-------------------------------------------*/
proc sgplot data=accessibility_scores_tiered;
    histogram accessibility_score / binwidth=0.05;
    density accessibility_score;
    title "Distribution of Accessibility Scores (Tiered E2SFCA)";
run;

/*-------------------------------------------
  STEP 8: Survey Data Preparation
-------------------------------------------*/
data survey_data;
    set survey_data;
    weight_num = input(weight, best12.);
    weight_ab  = input(weightAB, best12.);
    weight_cd  = input(weightCD, best12.);
    weight_ef  = input(weightEF, best12.);
    weight_gh  = input(weightGH, best12.);
run;

%macro surveyfreq(var, wt);
proc surveyfreq data=survey_data;
    strata county_code;
    cluster case;
    weight &wt;
    tables colon_screening*&var / row;
run;
%mend;

%surveyfreq(screening_behavior, weight_num);
```

```
%surveyfreq(sex, weight_ab);
%surveyfreq(health_coverage, weight_num);
%surveyfreq(age_grp, weight_ab);
%surveyfreq(race, weight_num);
%surveyfreq(code_fips, weight_cd);

/*-------------------------------------------
  STEP 9: Logistic Models
-------------------------------------------*/
proc logistic data=survey_data descending plots=oddsratio;
    class age_grp(ref='18-29') sex(ref='1') insurancetype(ref='1')
          married(ref='1') income(ref='1') screening_behavior(ref='1')
          race(ref='1') code_fips(ref='34027') educ(ref='3')
          delayed_care(ref='2') fin_hard(ref='2') food_insecur(ref='1');
    model colon_screening = age_grp*screening_behavior*insurancetype / selection=backward;
    title '3-Way Interaction: Age Group x Screening Behavior x Insurance Type';
run;

data survey_binary1;
    set survey_data;
    colon_screen = (colon_screening > 1);
run;

data survey_binary2;
    set survey_data;
    full_adhere = (colon_screening = 1);
run;

proc logistic data=survey_binary2;
    class insurancetype(ref='1') screening_behavior(ref='1');
    model full_adhere(event='1') = insurancetype screening_behavior insurancetype*screening_behavior;
    title 'Binary Logistic Model with Interaction';
run;

proc logistic data=survey_binary2;
    class insurancetype screening_behavior age_grp sex code_fips (param=ref);
    model full_adhere(event='1') =
        insurancetype screening_behavior insurancetype*screening_behavior
        / selection=stepwise;
    output out=predicted_values p=pred_prob;
```